

EIN092B: Visualización - Sesión 7
Laboratorio de Técnicas de Agrupación de Datos
Guía de Actividades - Sesión Evaluada

11 de septiembre de 2026

1. Objetivos

- Explorar conjuntos de datos usando técnicas de agrupamiento (*clustering*) para descubrir sub-grupos no conocidos previamente.
- Entender los conceptos y mecanismos asociados a las técnicas K-Means, DBSCAN y GMM, resaltando las ventajas, desventajas y supuestos de cada una.
- Utilizar técnicas de reducción de dimensionalidad vistas en la sesión anterior como herramienta de preprocesamiento y visualización para evaluar y comparar los agrupamientos obtenidos.

2. Indicaciones generales.

Las actividades planteadas en esta guía deben ser entregadas durante la sesión de laboratorio.

2.1. Sobre el proceso de revisión de actividades.

Para el proceso de revisión consideren lo siguiente:

- La revisión y evaluación de cada una de las actividades se realizará en dos etapas: (i) una validación básica de requerimientos funcionales mínimos durante la sesión, y (ii) una verificación de requerimientos funcionales y no funcionales posterior a la sesión.
- Durante la sesión, avise al staff (profesor o ayudante) cuando complete una o varias actividades para que muestre y explique sus resultados, y responda las preguntas que se le realizarán sobre los conceptos asociados.
- Cualquier integrante del grupo debe ser capaz de responder las preguntas. Si las respuestas no son satisfactorias, o algún integrante no es capaz de responder, no se revisará la actividad hasta que demuestren que TODOS los integrantes del grupo demuestren un entendimiento mínimo sobre lo que están entregando.

- Durante la sesión, el staff marcará en una planilla cuando Ud. considere una actividad terminada y haya pasado la interrogación. El tener la actividad marcada como revisada en la sesión no entrega puntaje, y solo asegura que cumple con requerimientos para que entregue sus archivos y pase a la siguiente etapa de revisión. El formato de entrega de sus archivos se indica en la Sección 2.2.
- En el caso de las actividades no entregadas en la sección correspondiente, contará como no entregada a tiempo y **deberá entregar la actividad corregida en la siguiente sesión, aplicando descuentos por atraso (penalización correspondiente al 35 % del puntaje inicial si se entrega en la siguiente sesión. Si se entrega en dos sesiones posteriores, 50 %).**
- En el caso de que algún integrante del grupo no este presente cuando su grupo correspondiente presente la evaluación y su correspondiente interrogación, deberá presentarlo por su propia cuenta en las siguientes sesiones y sufriendo la penalización por atraso.

2.2. Indicaciones de formato para entrega.

La evaluación constará de dos partes. La primera parte consiste en una validación básica y preguntas sobre los códigos propuestos por el grupo. La segunda parte constará de una verificación de sus códigos y respuestas, la cual se realizará sobre los archivos que deberá entregar vía Aula USM, los cuales deben seguir estrictamente las especificaciones que se le indican a continuación:

1. Debe crear una carpeta con el nombre *GXX_Sesion7*, donde *XX* el número de grupo. Por ejemplo, la carpeta para el grupo 5 se llamaría *G5_Sesion7*. El número de su grupo fue definido en sesiones anteriores.
2. Dentro de esta carpeta, debe incluirse un archivo de Jupyter Notebook que contenga el desarrollo de las actividades y las respuestas correspondientes.

Recordar que **será responsabilidad del grupo entregar de manera correcta los archivos**. Sin embargo, solo se revisarán las actividades marcadas como revisadas durante la sesión. Cualquier actividad adicional no será considerada en la revisión.

3. Introducción

En la sesión anterior se estudiaron técnicas de *reducción de dimensionalidad y visualización* (PCA, t-SNE y UMAP), orientadas a encontrar representaciones de baja dimensión que faciliten explorar visualmente la estructura de un conjunto de datos. En esta sesión, el foco cambia hacia el *agrupamiento* o *clustering*: dado un conjunto de características $X_1, X_2, \dots, X_p$ medidas sobre n observaciones, y sin contar con una etiqueta de clase, el objetivo es particionar las observaciones en subgrupos tales que las observaciones dentro de un mismo grupo sean similares entre sí, y distintas de las observaciones en otros grupos.

Existen múltiples familias de algoritmos de clustering, que difieren en los supuestos que hacen sobre la forma de los grupos, la necesidad de especificar a priori el número de grupos, y su capacidad de manejar outliers. En esta guía se exploran tres enfoques con supuestos muy distintos entre sí:

- **K-Means:** Un método basado en distancia al centroide, que asume implícitamente grupos de forma esférica y de tamaño similar, y requiere especificar el número de clusters K de antemano.
- **DBSCAN (*Density-Based Spatial Clustering of Applications with Noise*):** Un método basado en densidad, capaz de encontrar grupos de forma arbitraria y de identificar outliers como ruido, sin necesidad de especificar el número de clusters a priori [ester1996density].
- **GMM (*Gaussian Mixture Models*):** Un método probabilístico que modela los datos como una mezcla de distribuciones gaussianas, permitiendo una asignación “suave” (probabilística) de cada observación a los distintos grupos, en lugar de una asignación “dura” (única) como en K-Means.

En varias de las actividades de esta guía se utilizará PCA como paso de preprocesamiento antes de aplicar las técnicas de clustering, tal como se hizo en la sesión anterior con t-SNE y UMAP. Esto no solo permite visualizar los resultados en 2D, sino que en el caso de técnicas como DBSCAN resulta prácticamente indispensable, dado que las nociones de distancia y densidad en las que se basa este algoritmo pierden significado en espacios de muy alta dimensionalidad.

4. Actividades

En las siguientes subsecciones se describen las actividades a realizar. Todas las actividades de esta guía se desarrollan sobre un mismo conjunto de datos, el dataset *Fashion-MNIST*, conformado por 60.000 imágenes de artículos de vestuario. Cada imagen está representada con una profundidad de pixel de 28×28 y asociada a una etiqueta de entre 10 clases disponibles. Se emplearán distintas técnicas de agrupamiento sobre este conjunto de datos, con el fin de comparar sus resultados, supuestos y limitaciones.

4.1. K-Means Clustering (35 puntos)

K-Means es un enfoque simple para particionar un conjunto de datos en K distintos *clusters* que no se solapan. Cada cluster tiene un centroide que corresponde al promedio de puntos en ese grupo. El objetivo de K-Means es encontrar los K *clusters* que agrupan las observaciones similares entre sí, usando como criterio la distancia al centroide más cercano. Luego, actualizar los centroides como el promedio de los puntos asignados a cada cluster. Este proceso se repite hasta que las asignaciones no cambien o el movimiento de los centroides sea mínimo.

Una debilidad de K-Means es la sensibilidad a la inicialización de los centroides, donde una mala inicialización puede conducir a soluciones subóptimas. Por lo tanto, se han propuesto variantes para mitigar la inicialización (por ejemplo, K-Means++). Para mayores detalles del funcionamiento de K-Means, se provee una presentación en Aula USM.

Si bien existen distintas métricas para evaluar el agrupamiento realizado, como las métricas homogeneidad, completitud, inercia o el índice de *Silhouette*. En este laboratorio, evaluaremos la calidad de la agrupación por *clusters* usando inercia.

El algoritmo de K-Means requiere la integración de algunos pasos generales:

1. Seleccionar el número de *clusters* y asignar aleatoriamente K observaciones del conjunto de datos como centroides.
2. Iterar hasta que las asignaciones de *clusters* no cambien.
 - a) Asignar todos los puntos al centroide más cercano. Seguidamente, para cada uno de los K *clusters* calcule el centroide.
 - b) Asignar cada observación al *clusters* con el centroide más cercano (aquí más cercano es definido usando la distancia euclidiana).

Actividades a realizar en esta subsección:

- Implemente el algoritmo K-Means usando la librería *scikit-learn*. En este caso, conocemos que el dataset tiene 10 clases, por lo que se espera que el número óptimo de *clusters* sea $K = 10$. De todas formas, esto se hizo intencionalmente para guiar el laboratorio, y poder comparar el agrupamiento obtenido con K-Means con la referencia.
- Realizar un benchmarking utilizando K-Means con una inicialización aleatoria y con una inicialización `KMeans++`. Además, use K-Means con una etapa previa de reducción de la dimensionalidad usando PCA.

Debe considerar que K-Means es un método estocástico por lo que se recomienda ejecutar la inicialización más de una vez, en este caso establezca el parámetro `n_init` en 4. Por el contrario, PCA es un método determinista por lo que puede establecer `n_init` en 1.

```

1 from sklearn.cluster import KMeans
2 from sklearn.decomposition import PCA
3
4 kmeans = KMeans(init="k-means++", n_clusters=n_classes, n_init=4,
5               random_state=0)
6 kmeans = KMeans(init="random", n_clusters=n_classes, n_init=4, random_state
7               =0)
8 pca = PCA(n_components=n_classes).fit(data)
9 kmeans = KMeans(init=pca.components_, n_clusters=n_classes, n_init=1)

```

Para evaluar el ajuste de los clusters use como referencia la suma de las distancias al cuadrado (*sum of squared errors*, SSE) de cada punto de datos a su centroide más cercano (atributo `kmeans.inertia_`).

- Evalúe los cluster obtenidos en PCA + K-Means visualizando los centroides o visualizando los grupos formados en 2D. En este caso, establezca el número de componentes de PCA en 2.
- Finalmente, para K-Means con inicialización aleatoria, pruebe con distintos valores de K y haga un gráfico donde se muestre el SSE vs. el número de clusters. Discuta los resultados.

4.2. DBSCAN (40 puntos)

DBSCAN (*Density-Based Spatial Clustering of Applications with Noise*) es un algoritmo de clustering basado en densidad [1]. A diferencia de K-Means, DBSCAN no requiere especificar de

antemano el número de clusters, y es capaz de encontrar grupos de forma arbitraria (no solo esférica), además de identificar explícitamente observaciones que no pertenecen a ningún grupo (ruido u *outliers*).

El algoritmo se basa en los siguientes hiperparámetros:

- `eps` (ϵ): El radio máximo dentro del cual se busca a los vecinos de un punto.
- `min_samples`: El número mínimo de vecinos (incluyendo el propio punto) que debe tener un punto dentro de su radio `eps` para ser considerado un *punto núcleo* (*core point*).
- `metric`: La métrica de distancia utilizada para determinar la vecindad de cada punto (por ejemplo, `euclidean`, `manhattan`, `cosine`, `minkowski`, entre otras). La elección de esta métrica debe ajustarse a la naturaleza de los datos, ya que `eps` se interpreta directamente en las unidades de la métrica seleccionada. Cabe destacar que, cuando `metric='minkowski'`, se utiliza también el hiperparámetro auxiliar `p` para definir el orden de la distancia de Minkowski.

A partir de estos hiperparámetros, cada punto del conjunto de datos se clasifica en una de tres categorías:

- **Punto núcleo:** Tiene al menos `min_samples` vecinos dentro de su radio `eps`.
- **Punto borde:** No es un punto núcleo, pero cae dentro del radio `eps` de algún punto núcleo.
- **Punto de ruido:** No es punto núcleo ni punto borde; no pertenece a ningún cluster (en *scikit-learn*, se le asigna la etiqueta `-1`).

Los clusters se forman conectando puntos núcleo que son vecinos entre sí (junto con sus puntos borde asociados), lo que permite que un cluster tome formas alargadas o irregulares, algo que K-Means no puede representar dado que siempre genera fronteras equivalentes a un diagrama de Voronoi alrededor de sus centroides.

Una consecuencia importante de que DBSCAN dependa de una noción de distancia y densidad, es que su desempeño se degrada en espacios de muy alta dimensionalidad, donde las distancias entre puntos tienden a volverse similares entre sí, dificultando distinguir qué tan “denso” es realmente un vecindario. Por este motivo, en esta sección aplicaremos DBSCAN sobre una representación de menor dimensionalidad obtenida mediante PCA, en lugar de sobre los datos originales.

Actividades a realizar en esta subsección:

1. Reduzca la dimensionalidad de los datos originales de Fashion-MNIST mediante PCA y aplique DBSCAN sobre esta representación usando la librería *scikit-learn*.

```
1 from sklearn.cluster import DBSCAN
2 from sklearn.decomposition import PCA
3
4 pca = PCA(n_components=30, random_state=0)
5 x_train_pca = pca.fit_transform(x_train)
6
7 dbscan = DBSCAN(eps=3.0, min_samples=10)
8 labels = dbscan.fit_predict(x_train_pca)
```

Reporte cuántos clusters fueron encontrados y qué porcentaje de las observaciones fue etiquetado como ruido.

2. DBSCAN es considerablemente más sensible a la elección de sus hiperparámetros que K-Means. Realice un benchmarking variando `eps` (manteniendo `min_samples` fijo) y, de forma análoga, variando `min_samples` (manteniendo `eps` fijo en un valor razonable según el análisis anterior). Para cada configuración, reporte el número de clusters encontrados y el porcentaje de puntos etiquetados como ruido. Discuta el efecto de ambos hiperparámetros sobre los resultados, incluyendo qué sucede en los extremos: valores de `eps` muy pequeños o muy grandes, y valores de `min_samples` muy bajos o muy altos.
3. Realice un segundo benchmarking, esta vez variando el parámetro `metric`, manteniendo fijos `eps` y `min_samples` en valores razonables según los puntos anteriores. Tenga en cuenta que, al cambiar de métrica, es posible que deba también reajustar `eps`, ya que este se interpreta en las unidades de la métrica utilizada. Discuta qué diferencias observa en los clusters obtenidos y qué métrica le parece más adecuada para este conjunto de datos.
4. Una heurística común para elegir un valor razonable de `eps` consiste en calcular, para cada punto, la distancia a su k -ésimo vecino más cercano (con $k = \text{min_samples}$), ordenar estas distancias de menor a mayor, y graficarlas. Un “codo” en este gráfico sugiere un valor adecuado de `eps`. Implemente esta heurística utilizando `NearestNeighbors` de *scikit-learn* y compare el valor sugerido con los valores utilizados en los puntos anteriores.

```
1 from sklearn.neighbors import NearestNeighbors
2 import numpy as np
3
4 k = 10 # equivalente a min_samples
5 neighbors = NearestNeighbors(n_neighbors=k).fit(x_train_pca)
6 distances, _ = neighbors.kneighbors(x_train_pca)
7 k_distances = np.sort(distances[:, -1])
8
9 plt.plot(k_distances)
10 plt.ylabel(f"Distancia al {k}-ésimo vecino más cercano")
11 plt.xlabel("Puntos ordenados")
```

5. Visualice los clusters obtenidos (incluyendo los puntos de ruido) en 2D. Para esta visualización, tiene libertad de elegir la técnica de reducción de dimensionalidad que estime más adecuada entre las vistas en la sesión anterior, fundamentando su elección e implementación. Finalmente, compare esta visualización con la obtenida para K-Means en la subsección anterior y discuta la competencia de los algoritmos para formar clusters que coincidan con las 10 clases reales del conjunto de datos.

4.3. Gaussian Mixture Models (25 puntos)

Los *Gaussian Mixture Models* (GMM) son un enfoque probabilístico para clustering, que modela el conjunto de datos como generado a partir de una mezcla de K distribuciones gaussianas multivariadas, cada una con su propia media y matriz de covarianza. A diferencia de K-Means, que realiza una asignación *dura* (cada punto pertenece a un único cluster), GMM realiza una asignación *suave*: para cada observación, el modelo entrega la probabilidad de que dicha observación pertenezca a cada una de las K componentes de la mezcla.

Los parámetros del modelo (las medias, covarianzas y pesos de cada componente) se estiman mediante el algoritmo *Expectation-Maximization* (EM), que itera entre (i) estimar, dados los parámetros actuales, la probabilidad de pertenencia de cada punto a cada componente (paso E), y (ii) actualizar los parámetros de cada componente usando estas probabilidades como ponderadores (paso M), hasta alcanzar convergencia.

Un aspecto relevante de GMM es el parámetro `covariance_type`, que controla qué tan flexible puede ser la forma de cada componente gaussiana:

- `spherical`: Cada componente es esférico (equivalente, en cierto sentido, al supuesto implícito de K-Means).
- `diag`: Cada componente puede ser elíptico, pero con ejes alineados a los ejes originales.
- `tied`: Todos los componentes comparten una misma matriz de covarianza (misma forma y orientación, distinta posición).
- `full`: Cada componente puede tener una matriz de covarianza completamente general (elipses de cualquier orientación y forma).

A mayor flexibilidad, mayor es también el número de parámetros a estimar y, por lo tanto, el riesgo de sobreajuste con pocos datos.

Además de `covariance_type`, otros hiperparámetros relevantes de la implementación de *scikit-learn* son:

- `n_components`: El número de componentes (clusters) de la mezcla, análogo al K de K-Means.
- `max_iter`: El número máximo de iteraciones del algoritmo EM. Al ser un método iterativo que optimiza una función de verosimilitud no convexa, es posible que el algoritmo no converja dentro del número de iteraciones por defecto para conjuntos de datos complejos, por lo que puede ser necesario aumentarlo.
- `init_params`: El método usado para inicializar los parámetros del modelo antes de comenzar las iteraciones de EM. Por defecto, *scikit-learn* utiliza `'kmeans'`, es decir, ejecuta primero K-Means para obtener una inicialización razonable de las medias de cada componente. Una mala inicialización puede llevar a EM a converger a un óptimo local pobre, tal como ocurre con K-Means.
- `reg_covar`: Un término de regularización que se suma a la diagonal de cada matriz de covarianza estimada, para evitar que estas se vuelvan singulares (no invertibles) cuando hay pocos puntos o poca varianza en alguna dirección dentro de una componente. Valores muy pequeños pueden generar errores numéricos; valores muy grandes pueden sobre-suavizar las covarianzas estimadas.

Actividades a realizar en esta subsección:

1. Ajuste un modelo GMM con $K = 10$ componentes sobre la representación de Fashion-MNIST reducida con PCA, usando la librería *scikit-learn*.

```

1 from sklearn.mixture import GaussianMixture
2
3 gmm = GaussianMixture(n_components=10, covariance_type='full', random_state
    =0)
4 gmm.fit(x_train_pca)
5 labels = gmm.predict(x_train_pca)           # asignación dura (hard)
6 probs = gmm.predict_proba(x_train_pca)     # asignación suave (soft)

```

Obtenga la asignación dura de cada observación a un cluster (`predict`) y compárela, tanto visualmente como en términos del número de observaciones por cluster, con los resultados obtenidos anteriormente con K-Means.

2. Realice un benchmarking utilizando distintos valores de `covariance_type`, manteniendo $K = 10$. Visualice los clusters resultantes en 2D para cada caso y discuta cómo cambia la forma de los grupos encontrados. ¿Qué relación observa entre `covariance_type='spherical'` y los resultados obtenidos con K-Means?
3. Al igual que K-Means requiere elegir un valor de K , GMM también lo requiere. Sin embargo, al ser un modelo probabilístico, puede utilizar criterios estadísticos como el *Bayesian Information Criterion* (BIC) o el *Akaike Information Criterion* (AIC) para ayudar a seleccionar un valor adecuado.
 - Investigue brevemente qué miden BIC y AIC, y por qué ambos penalizan la complejidad del modelo en lugar de basarse únicamente en la verosimilitud del modelo sobre los datos de entrenamiento. ¿Qué problema podría ocurrir si simplemente eligiera el valor de K que maximiza dicha verosimilitud, sin ningún tipo de penalización? A continuación, ajuste modelos GMM para distintos valores de K y grafique el BIC (y opcionalmente el AIC) en función de K (atributos `gmm.bic(X)` y `gmm.aic(X)`). Compare esta curva con la curva de SSE vs. K obtenida para K-Means en la subsección anterior. ¿Se observa un patrón similar? ¿Qué valor de K elegiría según este criterio?
4. Seleccione un pequeño subconjunto de observaciones e inspeccione sus probabilidades de pertenencia (`predict_proba`) a cada componente. Identifique al menos una observación con una probabilidad claramente dominante hacia un único cluster, y al menos una observación cuyas probabilidades estén repartidas de forma más pareja entre dos o más clusters. Muestre las imágenes originales correspondientes a ambos casos y discuta: ¿qué tipo de información entrega esta asignación suave que no está disponible al usar K-Means o DBSCAN? ¿En qué aplicaciones prácticas podría ser valioso contar con esta información adicional?

Referencias

- [1] Martin Ester et al. "A density-based algorithm for discovering clusters in large spatial databases with noise". En: *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*. KDD'96. Portland, Oregon: AAAI Press, 1996, págs. 226-231.